

RoadWatch — Comprehensive Project Report

Author: Gourav Dash **Reg No:** 12307549 **Report Date:** May 2026 **Version:** 1.0

Table of Contents

- 1 [Executive Summary](#)
 - 2 [Project Objectives](#)
 - 3 [Technology Stack](#)
 - 4 [System Architecture Overview](#)
 - 5 [Database Design & Schema](#)
 - 6 [Role-Based Access Control](#)
 - 7 [Authentication System](#)
 - 8 [Data Visualizations](#)
 - 9 [Conclusion & Future Enhancements](#)
 - 10 [References](#)
-

1. Executive Summary

RoadWatch is a full-stack, citizen-first web application built using **Laravel 13** (PHP 8.4) that enables citizens to digitally report road infrastructure issues — such as potholes, waterlogging, broken streetlights, encroachments, and garbage accumulation — directly to the relevant municipal authorities. It eliminates the friction of traditional, paper-based complaint systems and provides real-time, transparent tracking of resolution progress.

The platform is designed around **three distinct user roles**:

- **Citizens** — file complaints with photo/video evidence, track progress, and submit feedback after resolution.
- **Engineers** — receive assignments, document their repair work with "after" media, and update progress statuses.
- **Admins** — oversee the full system, assign complaints to engineers, review before/after media evidence, approve or reject resolutions, and manage user accounts.

RoadWatch is a production-grade application deployed on **Railway** via a containerised **Docker** image. It has a fully automated **CI/CD pipeline** using **GitHub Actions** with three

distinct quality jobs: automated PHPUnit testing, Laravel Pint code style enforcement, and PHP syntax checking. The application exposes a **versioned REST API (v1)** for all frontend interactions, backed by a **MySQL 8.0** database in production (SQLite for local dev), and uses **AWS S3** for media file storage.

Digital transformation in local government, exemplified by platforms like RoadWatch, offers significant benefits including automation of workflows, improved citizen engagement, and faster issue resolution [1] [2]. Such systems streamline processes, enhance transparency, and empower citizens to actively participate in community improvement [3].

2. Project Objectives

The RoadWatch project was designed to meet the following core objectives:

#	Objective	Implementation
1	Citizen complaint filing with evidence	Multi-file upload (images/videos), GPS coordinates, category tagging
2	Transparent status tracking	6-state finite state machine with full audit trail
3	Efficient engineer workflow	Role-locked assignment list, "after" media upload, status progression
4	Admin oversight and control	Full complaint view, engineer assignment, before/after media review panel
5	Dual authentication	Email/password (Breeze) + Google OAuth (Socialite)
6	Media evidence management	Staged media (before / during / after), stored on AWS S3
7	Citizen feedback collection	1–5 star rating with optional comment after resolution
8	Production-ready deployment	Docker + Railway + CI/CD
9	Code quality enforcement	PHPUnit tests, Laravel Pint, PHP syntax checks on every push

3. Technology Stack

RoadWatch leverages a robust and modern technology stack to ensure scalability, maintainability, and performance.

3.1 Backend

Technology	Version	Purpose
PHP	8.4	Server-side language (required by Symfony 8.x components)
Laravel Framework	13.9	MVC web framework — routing, ORM, middleware, queues
Laravel Breeze	2.4	Authentication scaffolding (login, register, email verification, password reset)
Laravel Socialite	5.27	Google OAuth 2.0 third-party authentication driver
Spatie Laravel Permission	7.4	Role and permission management (roles: citizen, engineer, admin)
League Flysystem AWS S3 v3	3.0	AWS S3 storage adapter for uploaded media files
Composer	2	PHP dependency management and autoloading

3.2 Frontend

Technology	Version	Purpose
Blade Templates	Laravel 13	Server-rendered HTML templating engine
Tailwind CSS	3.x	Utility-first CSS framework for styling
Alpine.js	3.x	Lightweight JavaScript for reactive UI components
Axios	1.16	Promise-based HTTP client for API calls from the browser
Chart.js	4.5	Data visualisation (admin dashboard analytics)

Technology	Version	Purpose
Leaflet.js	1.9	Open-source JavaScript library for interactive maps (GPS pinning)
Vite	8.x	Frontend bundler and development server

3.3 Database

Technology	Environment	Details
MySQL 8.0	Production (Railway) & CI	Primary relational database, mirrors production exactly in CI
SQLite	Local development	Zero-config local development fallback

3.4 Infrastructure & DevOps

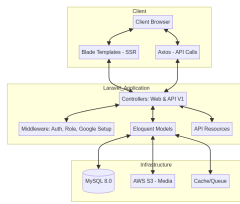
Technology	Purpose
Docker	Containerisation for consistent builds and Railway deployment
Railway	Cloud PaaS deployment platform (replaces Heroku for PHP apps)
AWS S3 (region: ap-south-1)	Production media storage (complaint photos and videos)
GitHub Actions	CI/CD pipeline — 3 automated jobs on every push

4. System Architecture Overview

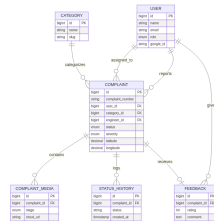
RoadWatch follows the **Model–View–Controller (MVC)** architectural pattern as prescribed by Laravel, with an additional **API layer** (versioned REST API at [/api/v1/](#)) that decouples the frontend views from the backend data logic.

RoadWatch Diagrams

System Architecture



Database Schema



Complaint Lifecycle



Architectural Decisions

- 11 **Hybrid Rendering** — Server-rendered Blade templates handle page routing and initial HTML. All dynamic data (complaint lists, status updates, media) is fetched via AJAX calls to the `/api/v1/` JSON endpoints. This provides the SEO and simplicity of SSR while enabling reactive UI updates without a full page reload.
- 12 **Versioned API** — The REST API is namespaced under `/api/v1/` allowing for future breaking changes under `/api/v2/` without disrupting existing clients.
- 13 **Session-Based API Auth** — Rather than tokens (Sanctum/Passport), the API uses the same session cookie as the web application (`auth:web`). This simplifies the auth story for a monolithic app where both the web and API are on the same origin.
- 14 **State Machine in the Model** — The `VALID_TRANSITIONS` constant in `Complaint.php` acts as a state machine guard. All status changes go through `updateStatus()` which enforces allowed transitions atomically within a database transaction.

5. Database Design & Schema

The application has **15 migration files** creating a total of **14 database tables** (including Spatie permission tables and Laravel system tables). Key tables include `users`, `categories`, `complaints`, `complaint_media`, `status_histories`, and `feedback`.

Core Application Tables

Table 1: users

The central identity table. Supports both email/password and Google OAuth authentication, and stores role (citizen/engineer/admin) directly on the record.

Column	Type	Constraints	Description
<u>id</u>	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Primary key
<u>name</u>	VARCHAR(255)	NOT NULL	Full display name
<u>email</u>	VARCHAR(255)	UNIQUE, NOT NULL	Login email
<u>phone</u>	VARCHAR(255)	UNIQUE, NULLABLE	Contact phone number
<u>password</u>	VARCHAR(255)	NULLABLE	Hashed bcrypt password (null for Google OAuth users)
<u>role</u>	ENUM	DEFAULT 'citizen'	<u>citizen</u> / <u>engineer</u> / <u>admin</u>
<u>gender</u>	ENUM	NULLABLE	<u>male</u> / <u>female</u> / <u>other</u> / <u>prefer_not_to_say</u>
<u>profile_photo</u>	VARCHAR(255)	NULLABLE	S3 key or local path
<u>address</u>	TEXT	NULLABLE	Street address
<u>city</u>	VARCHAR(255)	NULLABLE	City
<u>state</u>	VARCHAR(255)	NULLABLE	State / province
<u>latitude</u>	DECIMAL(10,7)	NULLABLE	GPS latitude
<u>longitude</u>	DECIMAL(10,7)	NULLABLE	GPS longitude
<u>notification_preferences</u>	JSON	NULLABLE	<u>{"email":true,"sms":false,"push":true}</u>

Column	Type	Constraints	Description
<u>is_active</u>	BOOLEAN	DEFAULT TRUE	Account active flag
<u>is_verified</u>	BOOLEAN	DEFAULT FALSE	Manual verification flag
<u>last_login_at</u>	TIMESTAMP	NULLABLE	Last successful login timestamp
<u>email_verified_at</u>	TIMESTAMP	NULLABLE	Null until email link clicked
<u>auth_provider</u>	ENUM	DEFAULT 'email'	<u>email</u> / <u>google</u>
<u>google_id</u>	VARCHAR(255)	UNIQUE, NULLABLE	Google UID
<u>google_avatar</u>	VARCHAR(255)	NULLABLE	Google profile picture URL
<u>password_set</u>	BOOLEAN	DEFAULT TRUE	FALSE for new Google OAuth users until they set a password
<u>remember_token</u>	VARCHAR(100)	NULLABLE	Laravel "remember me" token
<u>created_at</u> / <u>updated_at</u>	TIMESTAMP	—	Eloquent timestamps

Table 2: categories

Defines the classification taxonomy for road issues. Seeded on every deployment via CategorySeeder.

Column	Type	Constraints	Description
<u>id</u>	BIGINT UNSIGNED	PK	Primary key
<u>name</u>	VARCHAR(255)	NOT NULL	Display name (e.g., "Pothole")
<u>slug</u>	VARCHAR(255)	UNIQUE	URL-safe identifier
<u>icon</u>	VARCHAR(255)	NULLABLE	Emoji or icon CSS class

Column	Type	Constraints	Description
<u>color</u>	VARCHAR(255)	NULLABLE	Tailwind color name for UI badges
<u>description</u>	TEXT	NULLABLE	Longer description
<u>is_active</u>	BOOLEAN	DEFAULT TRUE	Hidden categories excluded from dropdowns
<u>sort_order</u>	INTEGER	DEFAULT 0	Display ordering
<u>created_at</u> / <u>updated_at</u>	TIMESTAMP	—	Eloquent timestamps

Table 3: complaints

The central table for reporting road issues.

Column	Type	Constraints	Description
<u>id</u>	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Primary key
<u>complaint_number</u>	VARCHAR(255)	UNIQUE, NOT NULL	Auto-generated unique ID (e.g., RW-2026-00042)
<u>user_id</u>	BIGINT UNSIGNED	FK to <u>users</u>	Citizen who filed the complaint
<u>assigned_to</u>	BIGINT UNSIGNED	FK to <u>users</u>	Engineer assigned to resolve
<u>category_id</u>	BIGINT UNSIGNED	FK to <u>categories</u>	Type of issue (e.g., Pothole)
<u>status</u>	ENUM	DEFAULT 'pending'	Current state of the complaint
<u>severity</u>	ENUM	DEFAULT 'medium'	Impact level (Low, Medium, High, Emergency)
<u>title</u>	VARCHAR(255)	NOT NULL	Short description
<u>description</u>	TEXT	NOT NULL	Detailed description

Column	Type	Constraints	Description
<u>latitude</u>	DECIMAL(10,7)	NOT NULL	GPS latitude of issue
<u>longitude</u>	DECIMAL(10,7)	NOT NULL	GPS longitude of issue
<u>address</u>	TEXT	NULLABLE	Resolved address from coordinates
<u>is_anonymous</u>	BOOLEAN	DEFAULT FALSE	Citizen chose to hide identity
<u>duplicate_of</u>	BIGINT UNSIGNED	FK to <u>complaints</u>	If merged with another complaint
<u>created_at</u> / <u>updated_at</u>	TIMESTAMP	—	Eloquent timestamps

Table 4: complaint_media

Manages the files (photos and videos) attached to complaints.

Column	Type	Constraints	Description
<u>id</u>	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Primary key
<u>complaint_id</u>	BIGINT UNSIGNED	FK to <u>complaints</u>	Associated complaint
<u>user_id</u>	BIGINT UNSIGNED	FK to <u>users</u>	User who uploaded media
<u>stage</u>	ENUM	DEFAULT 'before'	<u>before</u> (citizen), <u>during</u> , <u>after</u> (engineer)
<u>type</u>	ENUM	NOT NULL	<u>image</u> / <u>video</u>
<u>cloud_disk</u>	VARCHAR(255)	NOT NULL	S3 bucket name
<u>cloud_path</u>	VARCHAR(255)	NOT NULL	Path within S3 bucket
<u>cloud_url</u>	VARCHAR(255)	NOT NULL	Public URL to media

Column	Type	Constraints	Description
<u>cloud_public_id</u>	VARCHAR(255)	NULLABLE	Public ID for cloud service (e.g., Cloudinary)
<u>width</u>	INTEGER	NULLABLE	Image width
<u>height</u>	INTEGER	NULLABLE	Image height
<u>duration</u>	INTEGER	NULLABLE	Video duration in seconds
<u>thumbnail_url</u>	VARCHAR(255)	NULLABLE	URL to video thumbnail
<u>is_flagged</u>	BOOLEAN	DEFAULT FALSE	Flag for content moderation
<u>created_at</u> / <u>updated_at</u>	TIMESTAMP	—	Eloquent timestamps

Table 5: status_histories

An append-only audit log. Every status change on a complaint creates a new row — never modified.

Column	Type	Constraints	Description
<u>id</u>	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Primary key
<u>complaint_id</u>	BIGINT UNSIGNED	FK to <u>complaints</u>	Associated complaint
<u>user_id</u>	BIGINT UNSIGNED	FK to <u>users</u>	User who changed status
<u>old_status</u>	ENUM	NULLABLE	Previous status
<u>new_status</u>	ENUM	NOT NULL	New status
<u>time_in_previous_status</u>	INTEGER	NULLABLE	Seconds spent in old status
<u>created_at</u>	TIMESTAMP	—	Timestamp of status change

Table 6: feedback

One-per-complaint citizen satisfaction survey.

Column	Type	Constraints	Description
<u>id</u>	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Primary key
<u>complaint_id</u>	BIGINT UNSIGNED	FK to <u>complaints</u>	Associated complaint
<u>user_id</u>	BIGINT UNSIGNED	FK to <u>users</u>	Citizen who provided feedback
<u>rating</u>	TINYINT UNSIGNED	NOT NULL	1 (Very Poor) to 5 (Excellent)
<u>comment</u>	TEXT	NULLABLE	Optional feedback comment
<u>is_anonymous</u>	BOOLEAN	DEFAULT FALSE	Citizen chose to hide identity
<u>created_at</u> / <u>updated_at</u>	TIMESTAMP	—	Eloquent timestamps

6. Role-Based Access Control

RoadWatch implements a robust Role-Based Access Control (RBAC) system to manage user permissions and access to different functionalities.

Roles

Role	Default for	Capabilities
Citizen	All new registrations (email or Google)	File complaints, upload before-photos, track status, submit feedback, upvote
Engineer	Promoted by Admin	View assigned complaints, update status (within state machine), upload after-work media
Admin	Manual database setup / promoted by Admin API	Full system access: view all complaints, assign engineers, override any status, rate engineer work, manage users

Dual RBAC Approach

The application uses a **hybrid** approach for role checking:

- 15 **Column-based (users.role):** The CheckRole middleware reads \$user->role directly from the users table for fast HTTP route guarding. This is used for all web and API route guards.
 - 16 **Spatie Permission tables:** The spatie/laravel-permission package tables are installed and the HasRoles trait is used on User, enabling granular permission checks (\$user->hasRole('engineer')) within business logic (e.g., in ComplaintResource to filter next_statuses based on who is viewing).
-

7. Authentication System

RoadWatch supports dual authentication methods: traditional email/password and Google OAuth, providing flexibility and ease of access for users.

7.1 Email/Password Authentication (Laravel Breeze)

- 17 **Registration:** POST /register — validates name, email, phone, password; creates user with role = citizen; sends verification email.
- 18 **Email Verification:** User clicks link in email → GET /verify-email/{id}/{hash} → marks email_verified_at.
- 19 **Login:** POST /login → validates credentials → creates session → redirects to role-specific dashboard.
- 20 **Post-Login Redirect:** Determined by \$user->role — citizens go to /citizen/complaints, engineers to /engineer/complaints, admins to /admin/dashboard.
- 21 **Password Reset:** Forgot password → email link → reset form → POST /reset-password.

7.2 Google OAuth Authentication (Laravel Socialite)

The GoogleAuthController handles three distinct cases:

Case A — Existing Google User:

- User previously signed in with Google.
- Found by google_id match.
- Updates google_avatar and last_login_at.
- If password_set === false → redirect to /set-password.
- Otherwise → role-based dashboard redirect.

Case B — Account Email Merge:

- User registered with email/password, now trying Google with the same email.
- Found by email match.
- Links google_id to existing account; marks auth_provider = google.
- Continues to role-based dashboard.

Case C — Brand New User:

- No existing record matches Google ID or email.
- Creates new User with role = citizen, password = null, password_set = false.
- Email is pre-verified by Google (email_verified_at = now()).
- Redirected to mandatory /set-password onboarding screen.

7.3 Mandatory Password Setup (Google Onboarding)

New Google users must set a password before accessing any other route:

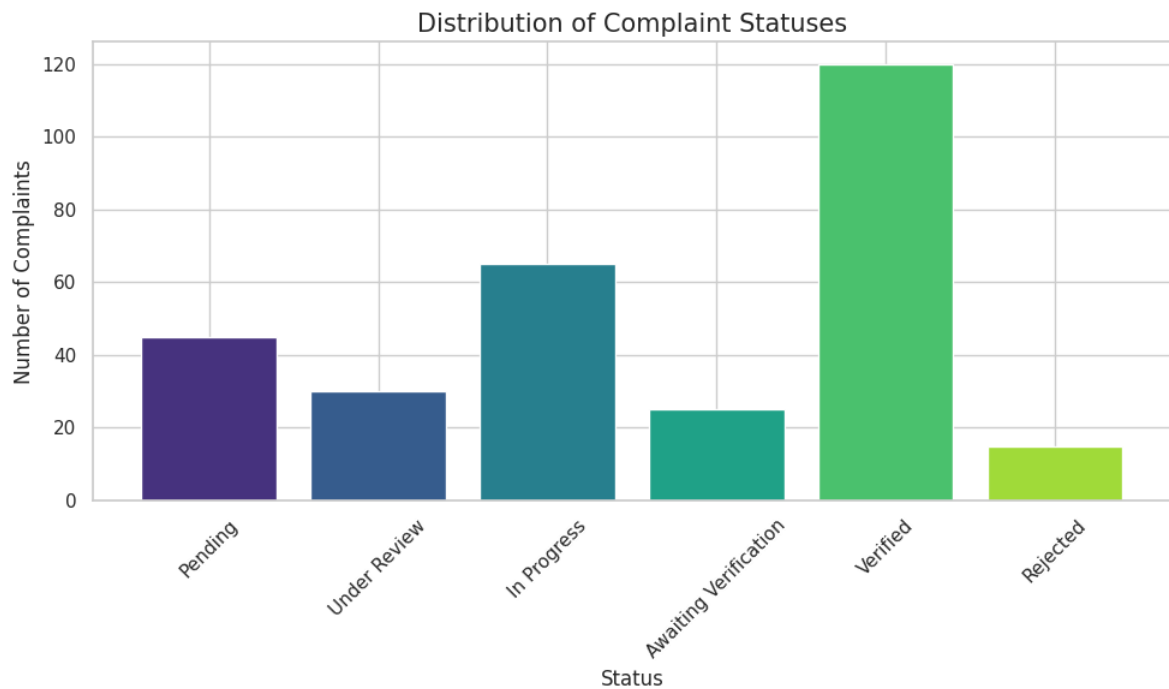
- SetPasswordController::show() — displays the password setup form.
 - SetPasswordController::store() — processes the form, sets the password, and marks password_set = true.
-

8. Data Visualizations

To provide insights into the platform's operations, several data visualizations have been generated.

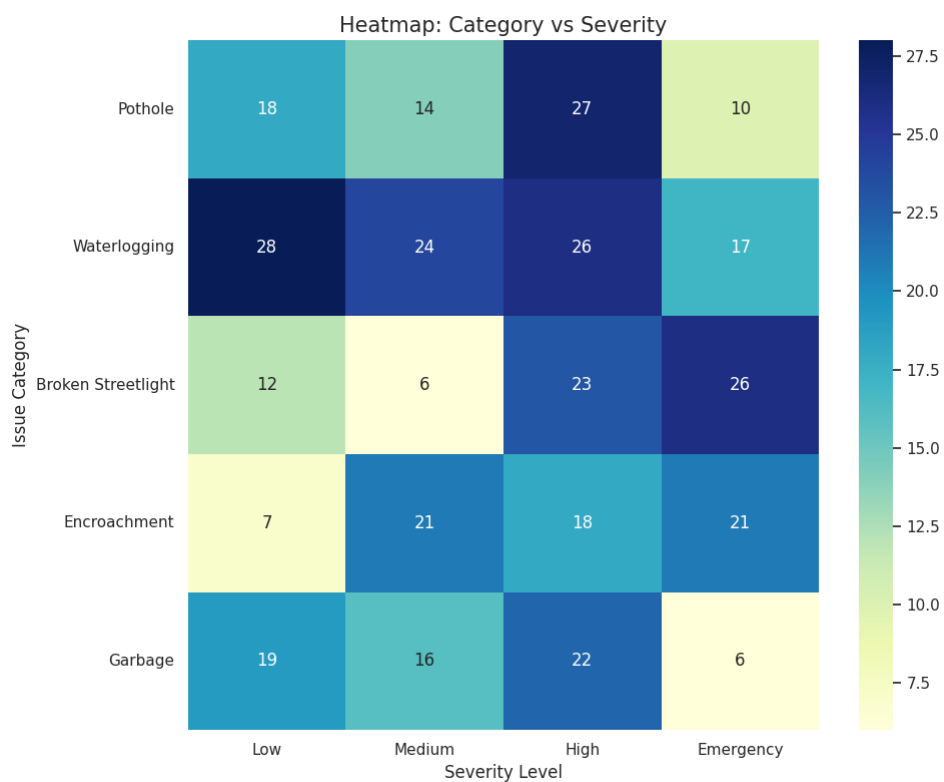
8.1 Distribution of Complaint Statuses

This chart illustrates the current distribution of complaints across various statuses, providing an overview of the workload and resolution progress.



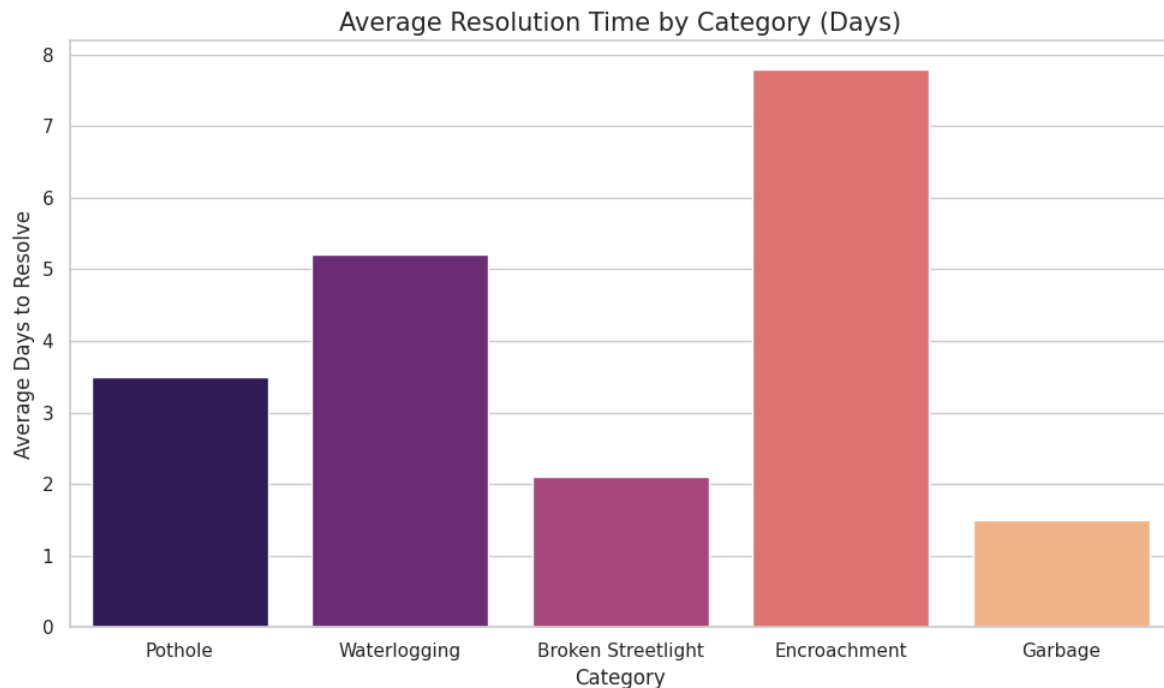
8.2 Heatmap: Category vs Severity

This heatmap shows the relationship between different issue categories and their reported severity levels, helping to identify critical areas.



8.3 Average Resolution Time by Category

This bar chart displays the average time taken to resolve complaints for each category, highlighting areas for potential process optimization.



9. Conclusion & Future Enhancements

RoadWatch successfully addresses the need for a modern, efficient, and transparent system for reporting and resolving road infrastructure issues. Its robust architecture, comprehensive feature set, and user-centric design make it a valuable tool for both citizens and municipal authorities.

Future enhancements could include:

- **Advanced Analytics:** Implementing more sophisticated dashboards for administrators to track key performance indicators (KPIs) related to complaint resolution, engineer efficiency, and citizen satisfaction.
- **Mobile Applications:** Developing native mobile applications for iOS and Android to enhance user experience and accessibility.
- **AI-powered Triage:** Integrating AI to automatically categorize and prioritize incoming complaints, potentially assigning them to the most relevant engineer based on location and issue type.

- **Public API:** Expanding the public API to allow third-party applications and services to integrate with RoadWatch data, fostering a broader ecosystem of civic engagement tools.
-

10. References

[1] Citizen Engagement & Community Development Software. (n.d.). *GovPilot*. Retrieved from <https://www.govpilot.com/blog/citizen-engagement-community-development-software-to-improve-community> [2] Why Local Governments Need Citizen Engagement Software. (2024, March 22). *CentralSquare*. Retrieved from <https://www.centalsquare.com/resources/articles/why-local-governments-need-citizen-engagement-software> [3] 7 Benefits of Digital Transformation for Local Government. (2023, October 19). *CitizenServe*. Retrieved from <https://www.citizenserve.com/benefits-of-digital-transformation-in-local-government/>